



ST. MARTIN'S ENGINEERING COLLEGE

(Autonomous Institution - UGC, Govt. of India)

[ESTD : 2002] [NAAC (A+) & NBA Accredited] [Affiliated to JNTUH]
[Approved by AICTE] [NIRF & ARIIA Ranked] [A Non Minority Institution]
Dhulapally, Near Kompally, Secunderabad - 500 100, Telangana.



ASSIGNMENT ANSWER BOOKLET

Hall Ticket No. : 24K81A0558

Name of the Student: THUMATI CHANDANA

Course : B-Tech

Branch : CSE-A

Year : I/II/III/IV

Sem : I / II

Assignment : I / II

Subject Name : Principles of Programming Language.

Last Date of Submission:

Actual Date of Submission :

MARKS AWARDED

Q.No.	a	b	Total
1			
2			
3			
4			
5			
Total			

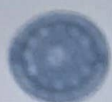
Chandana.

Signature of the Student

Signature of the Evaluator

Start Writing from Here

1. a) What is need for studying concepts of programming language
- A programming language is a formal language used to write instruction and algorithms for a computer. Studying programming language concepts means understanding the basic principles common to different programming languages rather than testing learning only the syntax of one language.
- Reasons for studying programming language concepts
1. Increased Capacity to Express Ideas
- The expressive power of a language affects the kinds of ideas and solutions that programmers can express. A programming language places limits on the control structures, and abstractions that programmers can use. Therefore, the forms of algorithms that can be constructed are also influenced by the language.



2. Improved Background for Choosing Appropriate Languages.

Programmers are often given a choice of languages for a new project. They may select the language they are most familiar with, even when another language is better suited to the problem. Knowledge of a wider range of languages & language constructs helps programmers select a language with features that best suit the particular problem.

3. Increased Ability to Learn New Languages.

Once the fundamental concepts of programming languages are understood, it becomes easier to learn a new language. The programmer can relate the concepts of the new language to concepts already known. For example, A programmer who knows the basic concepts of C can learn Java more easily, because both have concepts such as variables, datatypes, loops, & functions.

4. Better Understanding of Implementation.

Understanding implementation issues helps programmers know why programming languages are designed in particular ways and use their features more intelligently. It helps a programmer understand compilation & execution.

5. Better Use of Languages Already Known

Programming languages are large and complex, so programmers may not know all their features. Studying programming languages concepts helps them discover and use previously unknown features of languages they already know.

6 Overall Advancement of Computing

Studying programming languages concepts helps people make better decisions when choosing & developing programming languages. Popular languages are not always necessarily the best available languages.

1. b) Compare different programming domains and state their applications



A programming domain is a particular area in which computers are used to solve specific types of problems.

Different domains have different requirements, so programming languages have been developed with features suitable for those requirements.

1. Scientific Applications

Scientific applications were among the earliest uses of digital computers. They mainly require a large number of floating-point arithmetic computations. Array and matrices are commonly used as data structures, while counting loops & selections are common control structures.

Example: FORTRAN

Applications: Scientific calculations, mathematical computations, engineering calculations and scientific research.

2. Business Applications

Business applications mainly deals with processing business information. They require facilities for producing elaborate reports, storing decimal numbers and character data, and performing decimal arithmetic operations.

Example: COBOL

Applications: Banking, accounting, pay roll, financial processing

3. Artificial Intelligence

Artificial Intelligence is characterized by symbolic rather than numeric computations. Symbolic computations involves manipulating symbols or names rather than mainly performing numerical calculations.

Example: Lisp and Prolog.

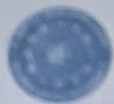
Applications: Artificial intelligence and logic-programming

4. Web Software

Web software is supported by a collection of languages, ranging from markup languages such as HTML to general-purpose languages such as Java. Languages such as JavaScript and PHP can be used to provide dynamic web content.

Applications: Websites, dynamic webpages & web applications

2.



Describe Context-free grammar (CFG) and BNF

(Backus - Naur Form) with example.

Context-free Grammar (CFG):

A context-free grammar (CFG) is a formal grammar used to describe the syntax of programming languages. It provides a systematic way of specifying how valid sentences and syntactic structures of a language can be formed.

Noam Chomsky developed context-free grammars in the mid-1950s. Among the different classes of grammars, context-free & regular grammars are useful for describing programming languages. Regular grammars can describe the forms of tokens, whereas context-free grammars can describe the syntax of whole programming languages, with minor exceptions.

Example

$\langle \text{expr} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle$

$\text{total} = \text{subtotal}_1 + \text{subtotal}_2$

Here, $\langle \text{expr} \rangle$ and $\langle \text{var} \rangle$ are nonterminal symbols, while subtotal_1 , subtotal_2 , total & $+$ are the elements used to form the expression.

Backus - Naur Form (BNF):

Backus - Naur Form (BNF) is a formal notation used for specifying the syntax of programming languages. John Backus introduced a formal notation for specifying programming language syntax, which was later modified by Peter Naur for the description of ALGOL 60. The revised method became known as Backus - Naur Form (BNF).

BNF became a popular method because it provides a concise and systematic way of describing programming language syntax. BNF is also called a metalanguage, because it is a language used to describe another language.



BNF Example :

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A | B | C$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle /$

$\langle \text{id} \rangle * \langle \text{expr} \rangle /$

$(\langle \text{expr} \rangle) | \langle \text{id} \rangle$

for example, the statement $A = B * (A + C)$

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$A = \langle \text{expr} \rangle$

$A = \langle \text{id} \rangle * \langle \text{expr} \rangle$

$A = B * (\langle \text{expr} \rangle)$

$A = B * (\langle \text{id} \rangle + \langle \text{expr} \rangle)$

$A = B * (A + \langle \text{id} \rangle)$

$A = B * (A + C)$

• This is called a leftmost derivation because the leftmost non terminal is replaced first at every step.

• The symbol '|' is used to represent alternatives, meaning logical OR

• Non-terminal symbols represent syntactic structures and written inside angular brackets, such as $\langle \text{expr} \rangle$ and $\langle \text{id} \rangle$

• Terminal symbols are actual symbols or tokens that appear in the program, such as $A, B, C, +, *$ and $=$.

BNF provides a simple and standard notation for writing those rules.

CFG provides the formal rules for describing the syntax of a programming language.



Q) Write a short notes on type binding and scoping with example

Type Binding

Binding is the association between an attribute and an entity, such as associating a variable with its type or value. The time at which a binding takes place is called binding time. Binding and binding times are important concepts in the semantics of programming languages.

Before a variable can be referenced in a program, it must be bound to a data type. The two important aspects of type binding are how the type is specified and when the binding takes place.

Types of Type Binding

1. Static Type Binding

In static type binding, the type of a variable is determined before or during program execution and generally remains fixed.

Example:

```
var sum = 0;
```

```
var total = 0.0;
```

```
var name = "Fred";
```

Here, sum is bound to int, total to float & name to string.

2. Dynamic Type Binding

In dynamic type binding, the type of a variable is determined during run time and can change during execution.

Example:

```
list = [10.2, 3.5];
```

```
list = 47;
```



Initially, list refers to an array of floating-point values. Later, it is assigned the integer value 47, so its type can change during execution.

Scoping

Scope is the range of statements in which a variable is visible. A variable is visible if it can be referenced or assigned in that statement. Scope rules determine how a particular name is associated with a variable.

Local variable: A variable declared within a program unit or block.

Non local variable: A variable that is visible in a program unit but declared outside it.

Global variable: A special category of nonlocal variable that can be visible to different parts of a program.

Example:

```
g = 3 # global
```

```
def sub1():
```

```
    a = 5 # local
```

Static Scoping

Static scoping determines the scope of a variable from the textual structure of the program. Therefore, its scope can be determined before program execution.

Example:

```
function big() {
```

```
    var x = 3;
```

```
    function sub2() {
```

```
        var y = x;
```

```
    }
```

```
}
```



Dynamic Scoping

Dynamic scoping is based on the calling sequence of subprogrammes, rather than their textual relationship. Therefore, the scope of a variable can be determined only during run time.

Example

$x = 10$

function A() {

$x = 20$

 B()

}

function B() {

 print(x)

}

A()

3. b) Explain short-circuit evaluation & Guarded commands

Short-circuit evaluation

It evaluates an expression is one in which the result is determined without evaluating all of operands and/or operators.

Example

$\text{if } (x \neq 0 \ \&\& \ y/x > 5)$

if $x \neq 0$ false, $y/x > 5$ is not evaluated.

Guarded Commands

Guarded commands are control structures where each statement has a Boolean condition called a guard.

Syntax:

if

 condition $\rightarrow$ statement

[] condition $\rightarrow$ statement

fi



Example :

```
if
i = 0 -> sum = sum + i
[] i > j -> sum = sum + j
[] j > i -> sum = sum + i
fi
```

if more than one condition is true, one corresponding statement can be selected non deterministically.

Q) What is over-loaded operator? Explain with example
An overloaded operator is an operator that has more than one meaning or operation, depending on the type of operands or the context in which it is used.

operator overloading allows the same operator symbol to perform different operations

Example

The operator "+" can be used for different purposes

In C++ :

```
int a = 10, b = 20;
```

```
cout << a + b;
```

Here, + performs arithmetic addition.

For strings :

```
string a = "Hello";
```

```
string b = "World";
```

```
cout << a + b;
```

Here, + performs string concatenation

thus, the same + operator performs :

- addition for numbers &
- concatenation for strings.



b) Discuss about control structures, selection statements and iterative statements

Control structures

A control structure controls the flow of execution of statements in a program. It provides way to select alternative paths and repeat statements.

selection statements

A selection statement chooses between two or more execution paths.

Types :

- Two-way selection : if-else
- Multiple selection : switch

Examples :

Two-way selection

```
if (marks >= 40)
    print ("Pass");
else
    print ("Fail");
```

Multiple selection

```
switch (choice) {
    case 1: print ("One");
            break;
    case 2: print ("Two");
            break;
}
```

Iterative statements

Iterative statements repeatedly execute a statement or group of statements.

Common types are

- while
- do-while
- for

Example :

```
for (i = 1; i <= 5; i++)
    print (i);
```

the loop executes repeatedly until the specified condition becomes false.